All Types of APIs
REST, SOAP, GraphQL, gRPC & WebSockets

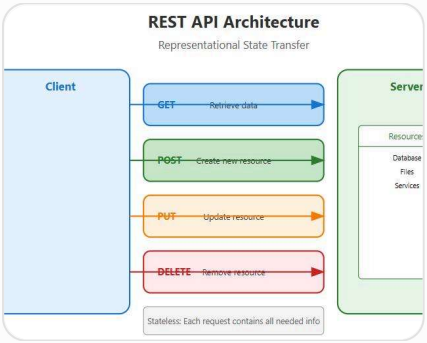
All Types of APIs Explained

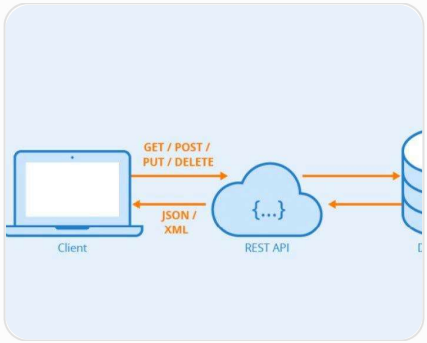
REST, SOAP, GraphQL, gRPC & WebSockets

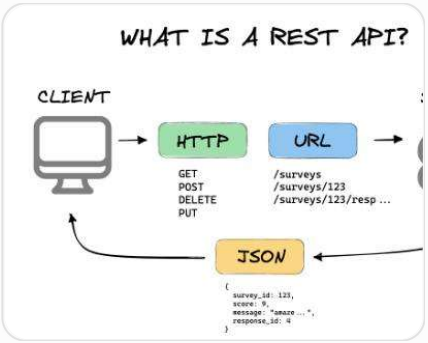
These are the five most commonly used communication technologies in modern software systems. Although people often group them together as "API types," **WebSockets are actually a communication protocol rather than a traditional request-response API style.**

---

1. REST API (Representational State Transfer)







What is it?

REST is the most widely used API architecture on the web.

It uses standard HTTP methods to communicate between client and server.

```
Client
   |
HTTP Request
(GET/POST/PUT/DELETE)
   |
Server
   |
JSON Response
```

HTTP Methods

Method	Purpose
GET	Read data
POST	Create
PUT	Update entire object
PATCH	Partial update
DELETE	Remove

Example

```
GET /users/10

Response

{
   "id":10,
   "name":"John"
}
```

Advantages

✅ Easy

✅ Human readable

✅ JSON support

✅ Browser friendly

✅ Cacheable

Disadvantages

❌ Over-fetching

```
Need only username

Returns

{
 id,
 username,
 email,
 phone,
 address,
 company,
 ...
}
```

❌ Under-fetching

Need user + orders

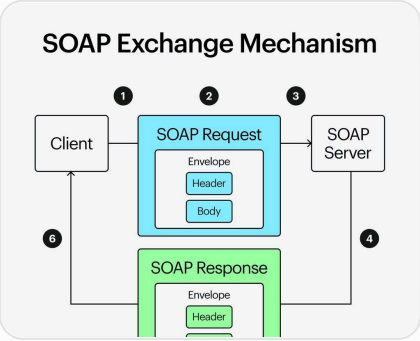
Need two API calls.

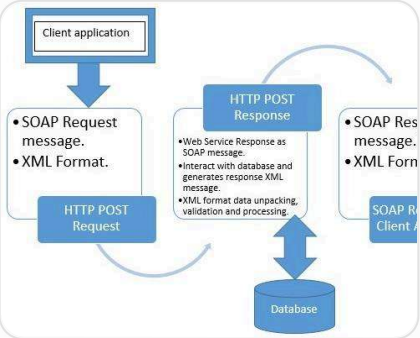
Used In

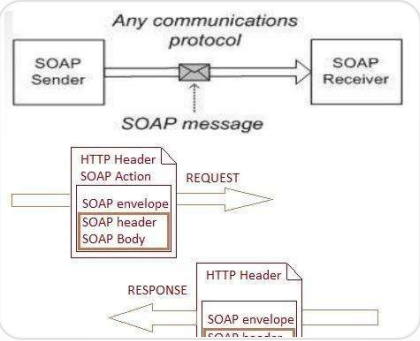
- Angular
- React
- Mobile Apps
- Public APIs

---

2. SOAP API







SOAP = Simple Object Access Protocol

Older enterprise communication protocol.

Uses XML only.

Example

```
<Envelope>
   <Body>
      <GetEmployee>
          <Id>10</Id>
      </GetEmployee>
   </Body>
</Envelope>
```

Features

- ✔ XML
- ✔ Strong standards
- ✔ WS-Security
- ✔ Transactions
- ✔ Reliable messaging

Advantages

Very secure

Strict contracts

Enterprise ready

Disadvantages

Large XML

Slow

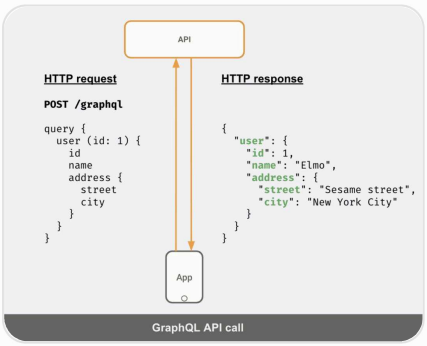
Harder to develop

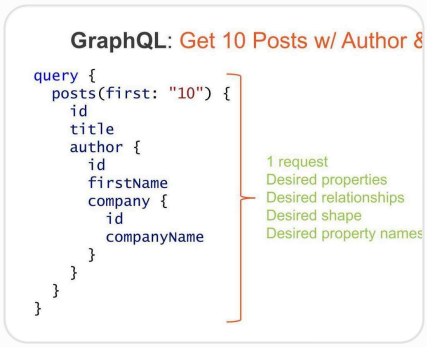
Used In

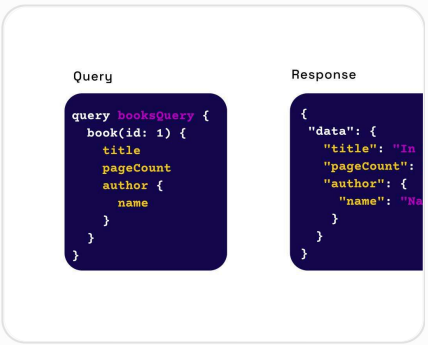
- Banking
- Government
- Insurance
- Legacy Enterprise Systems

---

3. GraphQL







Developed by Facebook.

Instead of many endpoints...

REST

```
/users

/orders

/products
```

GraphQL

```
/graphql
```

Only one endpoint.

Client decides what fields it needs.

Example

```
query{
 user(id:10){
   name
   email
 }
}
```

Response

```
{
"name":"John",
"email":"john@gmail.com"
}
```

Only requested fields returned.

Advantages

No over-fetching

No under-fetching

Single endpoint

Powerful queries

Disadvantages

Caching is harder

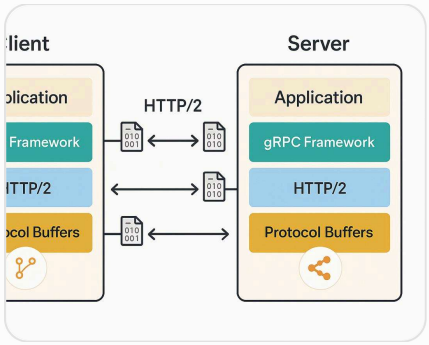
More complex backend

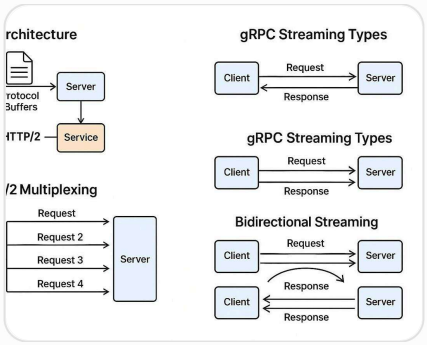
Used In

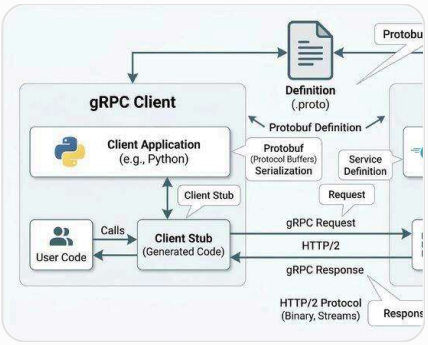
- GitHub API
- Shopify
- Modern SPAs

---

4. gRPC







Created by Google.

Uses

- HTTP/2
- Protocol Buffers (Binary)

Instead of JSON.

```
Client

↓

Serialize (ProtoBuf)

↓

HTTP/2

↓
```

```
Server
```

Example

```
service UserService{

 rpc GetUser(UserRequest)

 returns(UserResponse);

}
```

Advantages

Extremely fast

Binary protocol

Streaming

Strong typing

Auto-generated code

Streaming Types

```
Unary

Client → Server


Server Streaming

Client →→→ Server


Client Streaming

→→→ Client
```

```
Bidirectional

⇄⇄⇄
```

Disadvantages

Not browser friendly

Needs Proto files

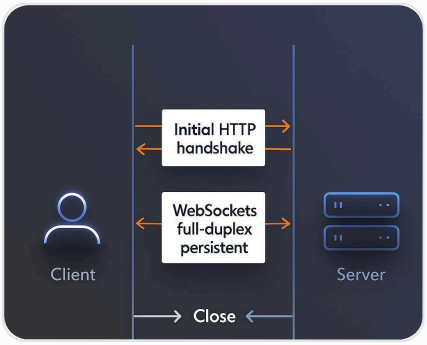
Hard to debug manually

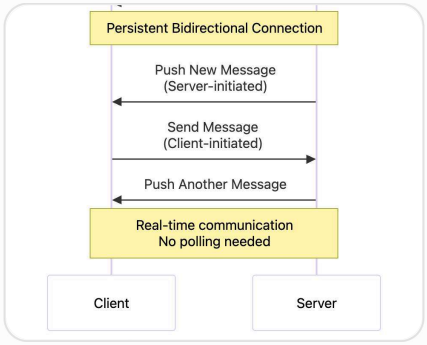
Used In

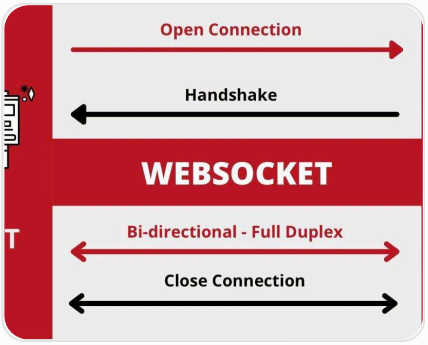
- Microservices
- Kubernetes
- AI systems
- High-performance backends

---

5. WebSockets







Unlike REST...

Connection stays open.

```
Client

⇅

Persistent Connection
```

```
⇅

Server
```

Real-time communication.

Example

```
Chat

↓

Send Message

↓

Instant Receive
```

Advantages

Real-time

Full duplex

Very low latency

Disadvantages

Not cacheable

Harder to scale

Persistent connections consume server resources

Used In

- Chat apps
- Live trading
- Gaming
- Notifications

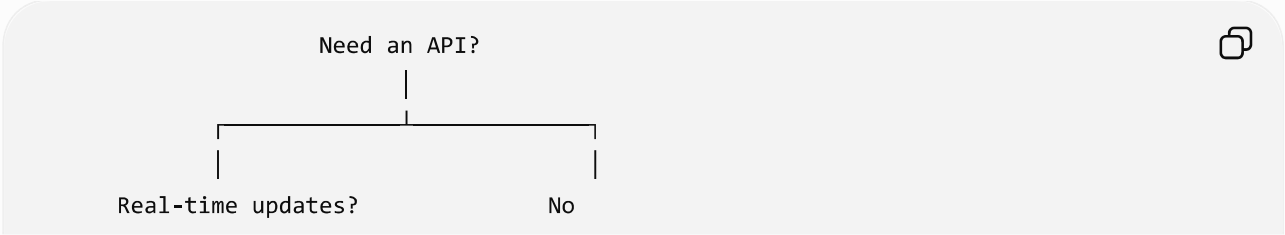
- Collaborative editors

Comparison Table

Feature	REST	SOAP	GraphQL	gRPC	WebSockets
Protocol	HTTP	HTTP/SMTP	HTTP	HTTP/2	WebSocket
Data Format	JSON	XML	JSON	Protocol Buffers	Any (JSON, binary, text)
Communication	Request–Response	Request–Response	Request–Response	Unary & Streaming	Full-duplex
Performance	Good	Slow	Good	Excellent	Excellent
Browser Support	Excellent	Good	Excellent	Limited (needs gRPC-Web for browsers)	Excellent
Type Safety	Low	High	High (Schema)	Very High	Depends on implementation
Caching	Excellent	Good	More complex	Limited	No
Best For	Public APIs, CRUD	Enterprise integrations	Flexible frontend APIs	Internal microservices	Real-time applications

---

When to Use Which?



```
           |                           |
          Yes                       CRUD API?
           |                           |
     Use WebSockets                  Yes
                                       |
                             Public web/mobile API?
                                       |
                                   Yes → REST
                                       |
                              Need flexible queries?
                                       |
                                 Yes → GraphQL
                                       |
                           Internal high-speed services?
                                       |
                                 Yes → gRPC
                                       |
                       Legacy enterprise integration?
                                       |
                                 Yes → SOAP
```

Real-world examples

- **REST**: Social media feeds, e-commerce product catalogs, weather APIs.
- **SOAP**: Core banking, insurance claims, government services.
- **GraphQL**: Dashboards and mobile apps where different screens need different data shapes.
- **gRPC**: Communication between microservices, machine learning inference services, Kubernetes control plane.
- **WebSockets**: Chat, multiplayer games, stock tickers, live collaboration, real-time notifications.

Rule of thumb

- Choose **REST** for most public APIs and standard CRUD applications.
- Choose **GraphQL** when clients need flexible data retrieval and want to avoid over-fetching.
- Choose **gRPC** for fast, strongly typed communication between backend services.
- Choose **SOAP** when integrating with legacy enterprise systems that require strict standards and advanced security.
- Choose **WebSockets** when the server and client need continuous, low-latency, bidirectional communication.

Do you like this personality?

ChatGPT can make mistakes. Check important info.